

PluginAny Routing Architecture: Upgraded Integration Testing Protocol

Objective: Validate the end-to-end integration of Valhalla route generation, dynamic Weather API variables, and Vehicle DB properties (specifically Rolling Resistance) into the FASTSim powertrain simulation.

Phase 1: Sub-System Validation (Isolated Component Tests)

Before passing data into the FASTSim physics engine, each external data source must be independently validated to ensure it formats data correctly for the physics bridge.

1.1 Vehicle Database Lookup (Rolling Resistance)

We now maintain a database of vehicle-specific aerodynamic and mechanical properties.

- **Test Action:** Query the vehicle DB for a specific `vehicle_id`.
- **Expected Output:** Ensure the API successfully retrieves the base `wheel_rr_coef` (Coefficient of Rolling Resistance, typically between 0.008 and 0.015) alongside standard parameters (mass, usable kWh, max kW).
- **Failure Condition:** If the DB returns `null` for `$C_{rr}$`, the system must fall back to a safe default (e.g., `0.012`) rather than crashing.

1.2 Weather API Integration Wrapper

- **Test Action:** Send a set of coordinates (lat/lon) to the Weather API wrapper.
- **Expected Output:** The wrapper must return standardized units:
 - `ambient_temp_c` (Celsius)
 - `wind_speed_kph` (km/h)
 - `wind_direction_deg` (0-360 degrees)
 - `precipitation_mm` (mm/hr)
- **Performance Gate:** This external API call must resolve in $< 150\text{ms}$. If it times out, the system must degrade gracefully (default to 25°C, 0 wind, 0 rain).

1.3 Valhalla Output Adapter

- **Test Action:** Request a standard route from the Valhalla engine.
- **Expected Output:** The adapter must correctly calculate `grade_pct` using Valhalla's elevation array and calculate the **compass heading** (0-360 degrees) for each route edge. The heading is strictly required for the headwind vector calculation in Phase 2.

Phase 2: The Physics Bridge & Math Logic (Integration Tests)

This is the most critical phase. The intern must verify the mathematical logic that adjusts the FASTSim payload based on the combined Valhalla and Weather data.

2.1 Aerodynamic Drag / Headwind Vector Math

The system cannot just add wind speed to vehicle speed. It must calculate the headwind vector.

- **Test Condition:** Vehicle speed is 80 km/h driving due North (Heading 0°). Wind is 20 km/h blowing from the North (Wind Direction 180°).
- **Expected Result:** The physics bridge calculates a direct headwind. The effective aerodynamic resistance applied to FASTSim should reflect 100 km/h.
- **Test Condition 2:** Crosswinds (e.g., Wind at 90°) should have a mathematically reduced impact on the frontal drag calculation.

2.2 Rolling Resistance (C_{rr}) Degradation

- **Test Condition:** The base C_{rr} from the vehicle DB is 0.010. The Weather API reports `precipitation_mm > 2.0` (heavy rain).
- **Expected Result:** The system applies a wet-road penalty multiplier (e.g., +15%) to the rolling resistance. The final `adjusted_rr_coef` injected into FASTSim should be 0.0115.

2.3 Auxiliary Power (HVAC Load) Estimation

- **Test Condition:** `ambient_temp_c` is 40°C.
- **Expected Result:** The system assumes a target cabin temp of 22°C. The physics bridge must inject an active `hvac_power_kw` load (e.g., 3.0 kW to 4.5 kW depending on the vehicle profile) into the simulation payload to represent the AC compressor running at full capacity.

Phase 3: End-to-End Payload & Simulation Tests

Once the logic is verified, the intern must test the unified API endpoint `POST /api/v1/physics/simulate` with the upgraded schema.

3.1 The "Perfect Conditions" Test

- **Action:** Send a payload with 25°C, 0 wind, 0 rain, and a fresh battery.
- **Verification:** The `final_soc` should match the baseline FASTSim output tested previously.

3.2 The "Worst-Case Scenario" Depletion Test

- **Action:** Send a payload explicitly designed to stress the powertrain:

JSON

```
{  
  "vehicle_id": "IN-2025-0007",  
  "environment": {
```

```

    "ambient_temp_c": 2.0,
    "wind_speed_kph": 30.0,
    "precipitation_mm": 5.0
  },
  "vehicle_state": {
    "starting_soc": 0.20,
    "protection_soc": 0.15,
    "hvac_power_kw": 4.0,
    "adjusted_rr_coef": 0.014
  },
  "route_edges": [
    {
      "edge_index": 0,
      "distance_m": 5000.0,
      "speed_kph": 90.0,
      "grade_pct": 3.0,
      "heading_deg": 180.0,
      "wind_direction_deg": 360.0,
      "start_coordinate": {"lat": 28.57, "lon": 77.05},
      "end_coordinate": {"lat": 28.60, "lon": 77.08}
    }
  ]
}

```

- **Verification:**

1. The API must return a **200 OK** status.
2. The **status** must be **depletion_triggered**.
3. The **depletion_coordinate** must trigger significantly *earlier* in the route distance than it would under perfect conditions, proving the environmental weather and tire variables are actively draining the battery in the simulation.

Phase 4: Production Deployment Checklist

Before merging this code to the staging branch, the intern must sign off on the following:

- [] **Valhalla Fallback:** If the FASTSim engine fails or times out, the Valhalla route coordinates are still returned to the client to ensure navigation UI does not break.
- [] **Weather API Caching:** Weather coordinates are grouped/cached via Redis for 10-15 minutes so we do not spam the weather provider API for every single route segment.
- [] **Logging:** Any missing `$C_{rr}$` values in the vehicle database trigger a silent backend warning log so the data team can patch the missing vehicle profile.